

DevOps, Agile, JIRA, Requirement Engineering and Traceability

1. Introduction

Modern software development requires organizations to deliver high-quality software quickly while continuously responding to changing customer requirements. Traditional software development methods often separate development, testing, operations, and business teams, which can lead to communication gaps, delays, and difficulty in managing changes.

Agile and DevOps are two approaches that help organizations improve software development and delivery. Agile focuses mainly on iterative development, customer collaboration, flexibility, and frequent delivery of working software. DevOps extends these ideas by bringing development and operations closer together and automating activities such as testing, integration, deployment, monitoring, and infrastructure management. Tools such as JIRA, Asana, and IBM Engineering Requirements Management DOORS help organizations plan projects, manage requirements, track tasks, and maintain traceability throughout the software development lifecycle.

2. DevOps

2.1 Definition of DevOps

DevOps is a combination of cultural philosophies, practices, and tools that helps organizations deliver applications and services at high speed and with greater reliability. According to AWS, DevOps increases an organization's ability to evolve and improve products faster than traditional software development and infrastructure-management approaches. The main idea behind DevOps is to remove barriers between development and operations teams. Instead of working as separate groups, developers, testers, security professionals, and operations teams collaborate throughout the application lifecycle.

2.2 Main Objectives of DevOps

1. Faster software delivery
2. Continuous integration and delivery
3. Better collaboration between teams
4. Automation of repetitive processes
5. Improved software reliability
6. Faster identification and resolution of defects
7. Continuous monitoring and feedback
8. Improved security
9. Reduced deployment risk
10. Better customer satisfaction

2.3 DevOps Lifecycle

1. Plan

Requirements are collected, prioritized, and converted into tasks or user stories.

2. Code

Developers write and modify application source code.

3. Build

The source code is compiled, packaged, and prepared for testing.

4. Test

Automated and manual tests are performed to identify defects.

5. Release

The tested software is prepared for deployment.

6. Deploy

The application is deployed to the required environment.

7. Operate

The application and infrastructure are maintained in the production environment.

8. Monitor

Application performance, availability, errors, logs, and infrastructure are monitored.

9. Feedback

Information from users, monitoring systems, and operations is provided back to the development team for further improvement.

2.4 Important DevOps Practices

Continuous Integration (CI)

Continuous Integration involves frequently integrating code changes into a shared repository. Automated builds and tests can be executed whenever changes are submitted.

Continuous Delivery (CD)

Continuous Delivery ensures that software is kept in a deployable state and can be released efficiently.

Infrastructure as Code (IaC)

Infrastructure is defined and managed using configuration files or code rather than manual configuration.

Continuous Monitoring

Applications and infrastructure are continuously monitored to detect performance issues, failures, and security problems.

Automation

Repetitive activities such as testing, building, deployment, and infrastructure provisioning are automated.

2.5 Benefits of DevOps

- **Speed:** Faster development and delivery.
- **Rapid Delivery:** More frequent releases and faster bug fixes.
- **Reliability:** Automated testing and monitoring improve reliability.
- **Scale:** Automation helps organizations manage large systems.
- **Improved Collaboration:** Development and operations teams share responsibilities.
- **Security:** Security and compliance can be integrated into development and deployment processes.

3. JIRA

3.1 Introduction to JIRA

JIRA is a project and work-management tool developed by Atlassian. It is widely used by software teams to plan, track, manage, and report work.

JIRA supports Agile methodologies such as Scrum, Kanban, and hybrid Agile approaches.

- Product backlogs
- Sprint planning
- Scrum boards
- Kanban boards
- User stories
- Tasks
- Bugs
- Epics
- Roadmaps
- Reports
- Dashboards
- Workflow management

3.2 Common JIRA Work Items

Epic

An Epic represents a large body of work that can be divided into smaller user stories. Example: Online Food Ordering System.

Story

A Story represents a user requirement or feature. Example: As a customer, I want to search for restaurants so that I can find suitable restaurants.

Task

A Task represents a specific piece of work. Example: Create restaurant search API.

Bug

A Bug represents a defect in the software. Example: Search results disappear when the user applies a price filter.

3.3 JIRA Scrum Workflow

Product Backlog → Sprint Planning → Sprint Backlog → Development → Testing → Review → Done

JIRA's Scrum functionality supports backlog prioritization, sprint planning, estimation, progress tracking, and reporting.

4. Agile vs DevOps

Parameter	Agile	DevOps
Main focus	Software development	Development and operations
Primary goal	Flexible and iterative development	Fast and reliable software delivery
Scope	Mainly development process	Entire software delivery lifecycle
Team collaboration	Developers, testers, customers	Development, operations, testing, security, and other teams
Delivery	Frequent increments	Continuous/frequent delivery and deployment
Automation	Helpful but not mandatory	Strong emphasis on automation
Testing	Continuous testing within iterations	Automated testing integrated into delivery pipeline
Operations	Usually treated separately	Integrated with development
Monitoring	Less central	Important part of the lifecycle
Common tools	JIRA, Trello, Azure Boards	JIRA, Jenkins, Git, Docker, Kubernetes, AWS tools
Feedback	Customer feedback	Customer + operational + monitoring feedback

Agile helps teams build the right product through iterative development, while DevOps helps teams build, test, release, deploy, and operate the product efficiently.

Agile = Iterative development and customer-focused delivery

DevOps = Collaboration, automation, continuous delivery, and operations

5. Case Study: JIRA in a Real-World Project

Project: E-Commerce Application

Consider an e-commerce company developing a new mobile shopping application.

Problem

The company has multiple development teams working on different features. Without a centralized tracking system, it becomes difficult to determine which features are being developed, which bugs are unresolved, who is responsible for each task, what should be completed in the current sprint, and whether the project is progressing according to schedule.

JIRA-Based Solution

The company creates a JIRA project and divides the work into Epics, Stories, Tasks, and Bugs.

Example Epic

Epic: Customer Shopping Experience

- User registration
- Product search
- Product filtering
- Shopping cart
- Online payment
- Order tracking

Example User Story

As a customer, I want to search for products so that I can quickly find the products I need.

Tasks

- Design search interface
- Create search API
- Connect database
- Implement search filters
- Write unit tests
- Perform integration testing

Sprint

The team selects a set of stories from the backlog and places them into a sprint.

Result

- Clearly assign responsibilities
- Track progress
- Identify blocked work
- Monitor bugs
- Prioritize requirements
- Review sprint performance
- Maintain visibility across the project

6. Case Study: Asana in a Real Project

Project: Product Launch

Consider a company preparing to launch a new software product. The project involves the product, development, marketing, design, and sales teams.

Asana can be used to organize the launch into a project containing tasks, subtasks, milestones, deadlines, and responsible team members.

Task	Responsible Team	Status
Finalize product requirements	Product	Completed
Design application screens	Design	In Progress
Develop application	Development	In Progress
Prepare marketing material	Marketing	To Do
Perform testing	QA	To Do
Launch product	Management	To Do

Example Subtasks

The task 'Prepare marketing material' can contain: Create product description, Design social media posts, Create promotional video, Prepare email campaign, and Review content.

Benefits

- Organize work
- Assign responsibilities
- Set deadlines
- Track progress
- Manage dependencies

- Collaborate
- View project schedules

7. JIRA vs Asana

Feature	JIRA	Asana
Primary use	Software and Agile development	General project and work management
Scrum	Strong support	Limited compared with JIRA
Kanban	Strong support	Supported
User stories	Strong support	Can be represented using tasks
Bugs	Dedicated issue tracking	Can be tracked as tasks
Sprint management	Strong	Less development-focused
Developer workflow	Excellent	Good
Project planning	Excellent	Excellent
Timeline	Available	Strong timeline support
Best suited for	Software development teams	Cross-functional business/project teams

JIRA is particularly suitable for software-development teams following Agile methods, whereas Asana is useful for broader project coordination involving multiple business functions.

8. Writing Effective User Stories

8.1 Definition

A User Story is a short description of a requirement written from the perspective of the user.

Common format: As a [user], I want [feature], so that [benefit].

Example: As a customer, I want to reset my password so that I can regain access to my account.

8.2 Characteristics of a Good User Story

A good user story should follow the INVEST principles:

- I – Independent: The story should be as independent as possible from other stories.
- N – Negotiable: The story describes the requirement without unnecessarily prescribing the implementation.
- V – Valuable: The story should provide value to the user or business.
- E – Estimable: The development team should be able to estimate the work.
- S – Small: The story should be small enough to complete within an iteration.

- T – Testable: The story should have clear conditions that determine whether it has been successfully completed.

9. Acceptance Criteria

9.1 Definition

Acceptance criteria are specific conditions that a software feature must satisfy before the requirement can be considered complete and accepted by the customer or Product Owner.

Example User Story

As a customer, I want to reset my password so that I can access my account if I forget my password.

Acceptance Criteria

11. The user should be able to select 'Forgot Password.'
12. The system should request the registered email address.
13. The system should send a password-reset link to a registered email.
14. The reset link should allow the user to create a new password.
15. The system should reject an invalid or expired reset link.
16. The user should be able to log in using the new password.

Acceptance criteria make requirements measurable and reduce ambiguity between customers, developers, testers, and project managers.

10. Advanced Requirement Elicitation Techniques

Requirement elicitation is the process of discovering, understanding, documenting, and validating the needs of stakeholders.

10.1 Interviews

An interview is a direct discussion between a requirements engineer and stakeholders.

- Structured interviews
- Semi-structured interviews
- Unstructured interviews

Example: For a food delivery application, the analyst may interview customers, delivery partners, restaurant owners, and customer-support employees.

Advantages

- Detailed information
- Immediate clarification
- Useful for complex requirements
- Helps identify hidden problems

Limitations

- Time-consuming
- Interviewees may provide biased information
- Requires skilled questioning

11. Ethnography

Ethnography involves observing users in their real working environment to understand how they actually perform their tasks.

Instead of simply asking users 'How do you perform this task?', the analyst observes 'How do you actually perform this task in your workplace?'

Example

Suppose a company wants to develop software for hospital staff. An analyst may observe nurses during their normal work to understand how patient information is recorded, how staff access records, how information is transferred, where delays occur, which tasks are repetitive, and what information staff need immediately.

Advantages

- Reveals real user behavior
- Identifies hidden requirements
- Captures informal processes
- Useful when users cannot clearly explain their workflow

Limitations

- Time-consuming
- Requires access to the working environment
- Observer presence may influence user behavior

12. Comparison of Interviews and Ethnography

Parameter	Interviews	Ethnography
Method	Asking stakeholders questions	Observing users
Main source	User responses	Actual behavior
Time required	Moderate	Usually higher
Hidden requirements	May identify some	Very effective
User environment	Not necessarily required	Important
Best for	Understanding opinions and needs	Understanding real workflows

Using both methods together can provide a more complete understanding of requirements.

13. Requirement Traceability Matrix (RTM)

13.1 Definition

A Requirement Traceability Matrix (RTM) is a document used to map requirements to related development and testing activities.

It ensures that every requirement is properly documented, implemented, tested, and verified.

13.2 Example RTM

Requirement ID	Requirement	Design/Module	Test Case	Status
REQ-01	User registration	Authentication Module	TC-01	Passed
REQ-02	User login	Authentication Module	TC-02	Passed
REQ-03	Product search	Search Module	TC-03	Passed
REQ-04	Add product to cart	Cart Module	TC-04	In Progress
REQ-05	Online payment	Payment Module	TC-05	Not Tested

13.3 Types of Traceability

Forward Traceability: Requirement → Design → Code → Test

Backward Traceability: Test → Code → Design → Requirement

Bidirectional Traceability: Combines forward and backward traceability and ensures that every requirement is implemented and every implemented feature has a valid requirement.

13.4 Benefits of RTM

- Ensures requirement coverage
- Prevents requirements from being missed
- Helps identify unnecessary functionality
- Supports testing
- Simplifies impact analysis
- Helps manage requirement changes
- Supports compliance and audits

14. Tools for Requirement Management

Requirement management tools help teams capture, organize, prioritize, track, version, and trace requirements throughout the project lifecycle.

17. IBM Engineering Requirements Management DOORS
18. JIRA
19. Azure DevOps
20. Jama Connect
21. Polarion
22. Requirements management modules in enterprise ALM platforms

15. IBM Engineering Requirements Management DOORS

15.1 Introduction

IBM Engineering Requirements Management DOORS is a requirements-management solution designed to help organizations manage requirements throughout the development lifecycle.

- Requirement organization
- Requirement traceability
- Version management
- Baselines
- Configuration management
- Requirement relationships
- Collaboration

15.2 Requirement Traceability in DOORS

DOORS can connect related artifacts so teams can understand relationships between requirements and other project artifacts.

Example: Business Requirement → System Requirement → Design → Implementation → Test Case
This makes it easier to determine the impact of a requirement change.

15.3 Advantages of DOORS

- Centralized requirement management
- Strong traceability
- Version control
- Baseline management
- Change management
- Requirement hierarchy
- Support for complex projects
- Useful for regulated industries

16. Overall Relationship Between the Concepts

A typical software project can follow this flow:

Stakeholder Needs → Requirement Elicitation → User Stories → Acceptance Criteria → Product Backlog → JIRA / Project Management Tool → Agile Development → DevOps Practices → Build → Test → Release → Deploy → Monitor → Feedback → Requirement Updates

The RTM provides traceability across these stages, while tools such as JIRA, Asana, and IBM DOORS help teams manage different aspects of the process.

17. Conclusion

Agile and DevOps have become important approaches for modern software development. Agile provides an iterative and flexible approach to developing software, while DevOps extends collaboration and automation across development, testing, deployment, and operations. JIRA provides strong support for Agile software-development teams through backlogs, Scrum and Kanban boards, sprint planning, issue tracking, and reporting. Asana provides flexible project and task management capabilities that are useful for cross-functional projects. Effective requirement engineering is equally important. User stories and acceptance criteria convert stakeholder needs into clear and testable requirements. Advanced elicitation techniques such as interviews and ethnography help analysts discover both explicit and hidden requirements. Finally, the Requirement Traceability Matrix and dedicated requirements-management tools such as IBM Engineering Requirements Management DOORS help ensure that requirements remain connected to design, implementation, and testing throughout the software development lifecycle.

Together, these practices and tools improve communication, traceability, quality, flexibility, and delivery efficiency in software projects.

References

- 23. AWS – What is DevOps? <https://aws.amazon.com/devops/what-is-devops>
- 24. Atlassian – Jira Agile Project Management
- 25. Asana – Project Management Guide
- 26. IBM – Engineering Requirements Management DOORS